



دانشکده مهندسی برق و کامپیوتر
گروه مهندسی کامپیوتر

عنوان:

پیاده‌سازی یک فریمورک ماژولار برای شبکه‌های عصبی تمام‌متصل و طبقه‌بندی تصاویر
CIFAR-۱۰

استاد محترم:

دکتر مریم شعاران

دانشجو:

امیررضا میرزاجانی اسکویی

مرداد ۱۴۰۴

نام خانوادگی دانشجو: میرزاجانی اسکوئی			نام: امیررضا
عنوان: پیاده‌سازی یک فریمورک ماژولار برای شبکه‌های عصبی تمام‌متصل و طبقه‌بندی تصاویر CIFAR-۱۰			
استاد محترم: دکتر مریم شعاران			
مقطع تحصیلی: کارشناسی ارشد		رشته: مهندسی مکانیک	گرایش: طراحی کاربردی
دانشگاه: تبریز		دانشکده: مهندسی مکانیک	تعداد صفحات: ۱۸
کلیدواژه‌ها: یادگیری عمیق، شبکه عصبی تمام‌متصل، معماری ماژولار، پس‌انتشار (Backpropagation)، بهینه‌سازی Adam			
چکیده: در این پروژه، یک فریمورک نرم‌افزاری برای ساخت و آموزش شبکه‌های عصبی تمام‌متصل^۱ با معماری ماژولار از پایه پیاده‌سازی شد. هدف اصلی، درک عمیق اجزای داخلی یک شبکه عصبی، از لایه‌های محاسباتی پایه گرفته تا الگوریتم‌های بهینه‌سازی پیشرفته بود. این فریمورک شامل پیاده‌سازی لایه‌های خطی (Affine)، توابع فعال‌سازی (ReLU)، یک چارچوب آموزشی جامع (Solver)، و الگوریتم‌های بهینه‌سازی مدرن مانند SGD+Momentum و Adam می‌باشد. صحت عملکرد فریمورک از طریق اعتبارسنجی گرادینان عددی تأیید گردید. در ادامه، این فریمورک برای آموزش و ارزیابی مدل‌ها بر روی وظیفه طبقه‌بندی تصاویر دیتاست CIFAR-۱۰ به کار گرفته شد. نتایج تجربی نشان داد که الگوریتم‌های بهینه‌سازی تطبیقی مانند Adam همگرایی سریع‌تری نسبت به SGD ساده دارند. مدل نهایی که با استفاده از هاپرپارامترهای بهینه‌سازی شده آموزش دید، توانست به دقت نهایی ۵۳.۰٪ بر روی مجموعه داده آزمون دست یابد که کارایی عملی فریمورک را در حل یک مسئله واقعی به اثبات می‌رساند.			

^۱ Fully-Connected

۱- مقدمه

در دهه اخیر، شبکه‌های عصبی عمیق به ابزاری بنیادین در حل مسائل پیچیده هوش مصنوعی، به‌ویژه در حوزه بینایی کامپیوتر، بدل شده‌اند. فریمورک‌های نرم‌افزاری سطح بالا نظیر TensorFlow و PyTorch، با فراهم آوردن ابزارهای انتزاعی قدرتمند، فرآیند طراحی و آموزش این مدل‌های پیچیده را به میزان چشمگیری تسریع و تسهیل کرده‌اند. با این حال، این سطح از انتزاع، جزئیات محاسباتی و ریاضیاتی که در قلب این شبکه‌ها قرار دارد را از دید کاربر پنهان می‌سازد و می‌تواند منجر به درکی سطحی از فرآیندهای بنیادین یادگیری شود.

انگیزه اصلی این پروژه، کاوش در همین لایه‌های پنهان و کسب درکی عمیق از مکانیزم‌های داخلی یک شبکه عصبی است. بر این اساس، هدف اصلی پروژه، طراحی و پیاده‌سازی یک شبکه عصبی تمام‌متصل از پایه، با تمرکز ویژه بر معماری ماژولار تعریف شد. در این رویکرد، هر جزء محاسباتی شبکه از لایه‌های خطی گرفته تا توابع فعال‌سازی، به‌عنوان یک واحد مستقل با دو تابع forward و backward پیاده‌سازی می‌شود. این طراحی نه تنها جریان داده و انتشار گرادیان در شبکه را شفاف می‌سازد، بلکه انعطاف‌پذیری بالایی برای ساخت و آزمایش معماری‌های گوناگون فراهم می‌آورد.

در نهایت، کارایی و صحت عملکرد این فریمورک سفارشی، از طریق آموزش و ارزیابی آن بر روی مجموعه داده استاندارد CIFAR-10، که یک چالش شناخته‌شده در حوزه طبقه‌بندی تصاویر است، به صورت عملی مورد سنجش قرار گرفت. در ادامه این گزارش، ابتدا به تشریح دقیق‌تر روش کار و اجزای کلیدی فریمورک پیاده‌سازی شده پرداخته، سپس نتایج به دست آمده از آزمایش‌های مختلف را ارائه کرده و در پایان، به تحلیل و نتیجه‌گیری خواهیم پرداخت.

۲- روش شناسی

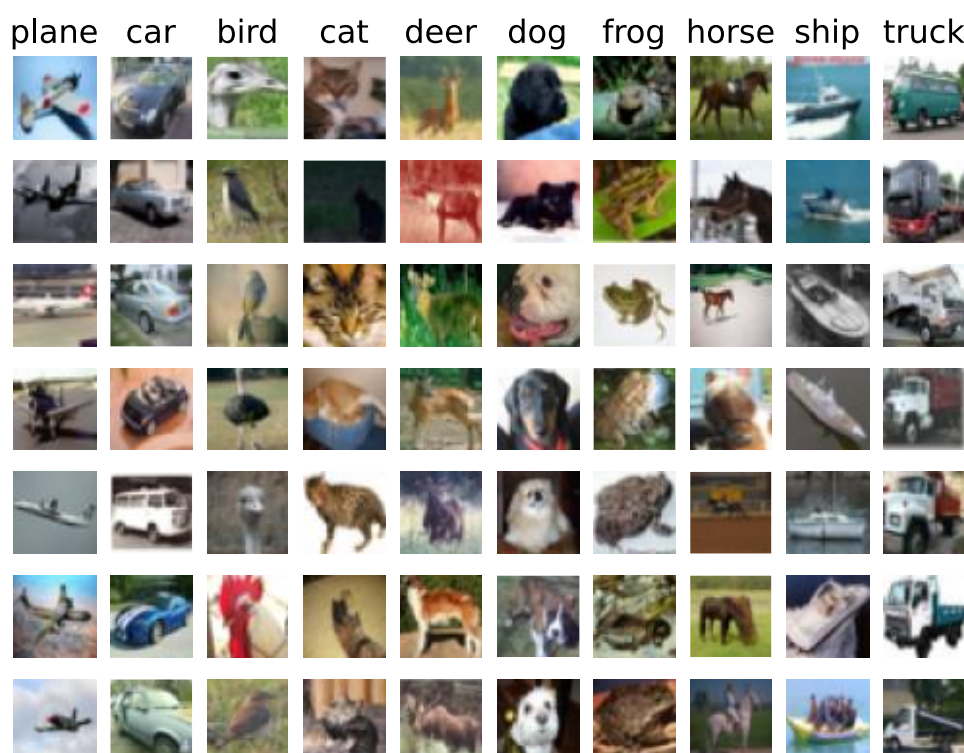
فریمورک پیاده‌سازی شده از چندین ماژول اصلی تشکیل شده است که هر کدام وظیفه مشخصی را بر عهده دارند.

۲-۱ مجموعه داده CIFAR-10 و پیش‌پردازش

مجموعه داده CIFAR-10 یکی از دیتاست‌های استاندارد و پرکاربرد در حوزه بینایی ماشین برای ارزیابی الگوریتم‌های طبقه‌بندی تصاویر است. این مجموعه داده شامل ۶۰,۰۰۰ تصویر رنگی با ابعاد کوچک 32×32 پیکسل می‌باشد. این تصاویر به ۱۰ کلاس مجزا تقسیم‌بندی شده‌اند که هر کلاس شامل ۶,۰۰۰ تصویر است. کلاس‌ها عبارتند از: هواپیما، اتومبیل، پرنده، گربه، گوزن، سگ، قورباغه، اسب، کشتی و کامیون. کل مجموعه داده به دو بخش اصلی تقسیم شده است: ۵۰,۰۰۰ تصویر برای مجموعه آموزشی و ۱۰,۰۰۰ تصویر برای مجموعه تست^۱ که امکان آموزش و ارزیابی مدل را فراهم می‌آورد.

پس از معرفی مشخصات کلی مجموعه داده CIFAR-10، در شکل ۱ نمونه‌هایی از تصاویر موجود در هر یک از ده کلاس نمایش داده شده است. همانطور که مشاهده می‌شود، این تصاویر با ابعاد کوچک 32×32

پیکسل، چالش‌های قابل توجهی را برای طبقه‌بندی به همراه دارند. تنوع بالا در زاویه دید، شرایط نوری، پس‌زمینه و موقعیت اشیاء در تصاویر، پیچیدگی مسئله را افزایش می‌دهد.



شکل ۱- نمونه تصاویر از مجموعه داده CIFAR-۱۰

جهت انجام فرآیندهای آموزش و ارزیابی، داده‌ها به سه زیرمجموعه مجزا تفکیک شدند: مجموعه آموزش (Training Set) شامل ۴۹,۰۰۰ تصویر، مجموعه اعتبارسنجی (Validation Set) شامل ۱,۰۰۰ تصویر، و مجموعه آزمون (Test Set) شامل ۱۰,۰۰۰ تصویر. مجموعه اعتبارسنجی نقشی کلیدی در فرآیند تنظیم فرارامترها ایفا می‌کند، در حالی که مجموعه آزمون صرفاً برای ارزیابی نهایی بهترین مدل انتخاب‌شده به کار می‌رود.

۲-۲ پیش‌پردازش داده‌ها^۱

به منظور آماده‌سازی داده‌ها برای ورود به شبکه، دو مرحله پیش‌پردازش اعمال گردید:

۱-۲-۲ برداری‌سازی^۲

هر تصویر، که در ابتدا به صورت یک تانسور $3 \times 32 \times 32$ (ارتفاع $\times$ عرض $\times$ کانال‌های رنگی RGB) نمایش داده می‌شد، به یک بردار ویژگی تک‌بعدی با ابعاد $D=3072$ تبدیل گردید. این فرآیند، که به آن "Flattening" نیز گفته می‌شود، برای انطباق با ساختار ورودی لایه‌های تمام-متصل ضروری است.

^۱ Data Preprocessing

^۲ Vectorization

۲-۲-۲ نرمال‌سازی با مرکزیت صفر^۱

به منظور استانداردسازی توزیع مقادیر پیکسل‌ها، یک تصویر میانگین از کل مجموعه آموزش محاسبه شد. سپس این تصویر میانگین از تمامی تصاویر موجود در مجموعه‌های آموزش، اعتبارسنجی و آزمون کسر گردید. این تکنیک به حذف بایاس‌های ناشی از شرایط نوری مشترک در تصاویر کمک کرده و فرآیند همگرایی الگوریتم بهینه‌سازی را تسریع می‌بخشد.

۳-۲ معماری ماژولار فریمورک

شالوده اصلی این پروژه بر پایه یک معماری ماژولار^۲ بنا نهاده شده است. این رویکرد در تضاد با طراحی‌های یکپارچه^۳ قرار دارد که در آن تمام منطق محاسباتی شبکه در یک تابع واحد و پیچیده محصور می‌شود. در طراحی ماژولار، هر جزء محاسباتی از شبکه به عنوان یک «لایه» مستقل و قابل استفاده مجدد در نظر گرفته می‌شود که وظایف مشخصی را بر عهده دارد.

هر لایه در این فریمورک از یک رابط برنامه‌نویسی استاندارد پیروی می‌کند که شامل دو تابع اصلی است:

- گذر مستقیم^۴: این تابع مسئولیت دریافت داده‌های ورودی از لایه قبلی، انجام مجموعه‌ای از عملیات ریاضی تعریف‌شده (مانند یک تبدیل خطی یا اعمال یک تابع غیرخطی)، و تولید خروجی برای لایه بعدی را بر عهده دارد. علاوه بر این، در حین این فرآیند، تمامی مقادیر میانی که برای محاسبه گرادیان‌ها در مرحله بعد ضروری هستند، در یک ساختار داده موقت به نام cache ذخیره می‌شوند.
- گذر معکوس^۵: این تابع، گرادیان تابع زیان نسبت به خروجی خود را (که به آن گرادیان بالادستی^۶ گفته می‌شود) به عنوان ورودی دریافت می‌کند. سپس با استفاده از مقادیر ذخیره‌شده در cache و با بهره‌گیری از قانون زنجیره‌ای، گرادیان تابع زیان را نسبت به ورودی‌ها و پارامترهای داخلی خود (مانند وزن‌ها و بایاس‌ها) محاسبه می‌نماید.

این ساختار علاوه بر جداسازی مسئولیت‌ها، چند ویژگی کلیدی دیگر نیز به فریمورک اضافه می‌کند:

- قابلیت ترکیب‌پذیری^۷: لایه‌ها می‌توانند به صورت زنجیره‌ای و با ترتیبات مختلف به یکدیگر متصل شوند تا معماری‌های متنوعی از شبکه ساخته شوند. این امر امکان آزمایش سریع ایده‌های جدید و مقایسه کارایی مدل‌های مختلف را فراهم می‌کند.
- مدیریت پارامترها: هر لایه مالک پارامترهای خود است و فریمورک ابزاری برای استخراج، تجمع و به‌روزرسانی این پارامترها در فرآیند آموزش ارائه می‌دهد.
- انعطاف‌پذیری و توسعه‌پذیری: افزودن یک لایه یا عملگر جدید تنها با تعریف کلاس یا تابع مربوط به آن انجام می‌شود و نیازی به بازنویسی سایر بخش‌های فریمورک نیست.

^۱ Zero-centering

^۲ Modular Architecture

^۳ Monolithic

^۴ Forward Pass

^۵ Backward Pass

^۶ Upstream Gradient

^۷ Composability

- سهولت در تست و اشکال زدایی: به دلیل استقلال لایه‌ها، می‌توان صحت عملکرد هر جزء را به صورت جداگانه مورد ارزیابی قرار داد (Unit Testing) که عیب‌یابی و نگهداری سیستم را ساده‌تر می‌سازد.
- یکپارچگی با فرآیند آموزش: این معماری بستری مناسب برای پیاده‌سازی الگوریتم‌های بهینه‌سازی و تکنیک‌های رگولاریزیشن فراهم می‌آورد و مراحل آموزش مدل را شفاف‌تر و قابل کنترل‌تر می‌کند.

در نتیجه، معماری ماژولار نه تنها خوانایی و قابلیت نگهداری کد را افزایش می‌دهد، بلکه امکان توسعه و ارزیابی سریع شبکه‌های عصبی متنوع را نیز مهیا می‌سازد. این ویژگی‌ها در بخش‌های بعدی، به‌ویژه در پیاده‌سازی لایه‌های محاسباتی بنیادین، ساختار مدل‌های شبکه و فرآیند آموزش و بهینه‌سازی نقش محوری ایفا خواهند کرد.

۲-۴ پیاده‌سازی لایه‌های محاسباتی بنیادین

در این فریمورک، هر لایه به عنوان یک ماژول مستقل تعریف شده است که دو عملیات کلیدی گذر مستقیم (Forward Pass) و گذر معکوس (Backward Pass) را پیاده‌سازی می‌کند. در ادامه، لایه‌های بنیادین مورد نیاز و فرمول‌های ریاضی مربوط به هر کدام معرفی می‌شوند:

۱. لایه خطی^۱

این لایه یک تبدیل خطی بر روی ورودی انجام می‌دهد:
گذر مستقیم:

$$Z = WX + B \quad (1-1)$$

که در آن

- $X \in \mathbb{R}^{d_{in} \times m}$: ورودی با (m نمونه در batch)

- $W \in \mathbb{R}^{d_{out} \times d_{in}}$: وزن‌ها

- $b \in \mathbb{R}^{d_{out} \times 1}$: بایاس

- $Z \in \mathbb{R}^{d_{out} \times m}$: خروجی

گذر معکوس:

با دریافت گرادیان بالادستی $\frac{\partial L}{\partial Z}$

$$\frac{\partial L}{\partial X} = W^T \frac{\partial L}{\partial Z}, \quad \frac{\partial L}{\partial W} = \frac{\partial L}{\partial Z} X^T, \quad \frac{\partial L}{\partial b} = \sum_{i=1}^m \frac{\partial L}{\partial Z} \quad (2-1)$$

۲. لایه فعال‌ساز^۲

این لایه‌ها نگاشت‌های غیرخطی بر روی داده‌ها اعمال می‌کنند. برخی از پرکاربردترین آن‌ها عبارت‌اند از:

^۱ Linear Layer / Fully Connected

^۲ Activation Functions

- سیگموئید (Sigmoid)

$$f(x) = \sigma(x) = \frac{1}{1+e^{-x}}, \quad f'(x) = \sigma(x)(1-\sigma(x)) \quad (3-1)$$

- تانژانت هایپربولیک (tanh)

$$f(x) = \tanh(x), \quad f'(x) = 1 - \tanh^2(x) \quad (4-1)$$

- ReLU

$$f(x) = \max(0, x), \quad f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \quad (5-1)$$

۳. لایه نرمال سازی

یکی از مهم ترین ها Batch Normalization است:

گذر مستقیم:

$$\mu = \frac{1}{m} \sum_{i=1}^m x_i, \quad \sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2 \quad (6-1)$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

که در آن پارامترهای γ و β قابل آموزش هستند.

گذر معکوس: فرمول ها نسبتاً پیچیده ترند و بر اساس قانون زنجیره ای از مشتقات μ ، $\hat{x}_i$ و μ محاسبه می شوند.

۴. لایه Softmax

این لایه معمولاً در خروجی برای مسائل دسته بندی استفاده می شود.

گذر مستقیم:

$$y_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (7-1)$$

گذر معکوس:

با دریافت گرادیان $\frac{\partial L}{\partial y}$

$$\frac{\partial L}{\partial z_i} = y_i \left(\frac{\partial L}{\partial y_i} - \sum_{j=1}^K y_j \frac{\partial L}{\partial y_j} \right) \quad (8-1)$$

۵. لایه زیان^۱

- Cross-Entropy Loss (رایج ترین برای دسته بندی):

$$L = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_{ik}^{(true)} \log(y_{ik}^{(pred)}), \quad \frac{\partial L}{\partial z} = y^{(pred)} - y^{(true)} \quad (9-1)$$

^۱ Loss Layer

- 'MSE Loss:

$$L = \frac{1}{2m} \sum_{i=1}^m \|y_i^{(pred)} - y_i^{(true)}\|^2, \quad \frac{\partial L}{\partial y^{(pred)}} = \frac{1}{m} (y^{(pred)} - y^{(true)}) \quad (10-1)$$

هر یک از این لایه‌ها به صورت یک ماژول مستقل در فریمورک پیاده‌سازی می‌شوند. با کنار هم قرار گرفتن آن‌ها می‌توان معماری‌های پیچیده‌تری از شبکه را ایجاد کرد. این پیاده‌سازی ماژولار موجب می‌شود که فرآیند آموزش (بخش ۲-۶)، بهینه‌سازی و اعمال تکنیک‌های رگولاریزیشن (بخش ۲-۷) به سادگی قابل توسعه و کنترل باشند.

۲-۵ ساختار مدل‌های شبکه

با بهره‌گیری از لایه‌های محاسباتی ماژولار که در بخش قبل معرفی و پیاده‌سازی شدند، در این پروژه دو نوع معماری شبکه عصبی تمام‌متصل طراحی و مورد استفاده قرار گرفت. این معماری‌ها به عنوان بدنه‌ی اصلی مدل‌ها عمل کرده و مبنای آزمایش، ارزیابی و مقایسه تکنیک‌های مختلف آموزش و رگولاریزیشن قرار گرفتند.

۲-۵-۱ مدل پایه: شبکه دو لایه

به منظور اعتبارسنجی اولیه پیاده‌سازی گذر مستقیم و معکوس و همچنین اطمینان از صحت عملکرد لایه‌های محاسباتی، یک شبکه عصبی ساده با دو لایه طراحی گردید. این شبکه شامل یک لایه ورودی، یک لایه مخفی و یک لایه خروجی است. ساختار ریاضی آن به شکل زیر تعریف می‌شود:

$$x \xrightarrow{\text{Affine}} h \xrightarrow{\text{ReLU}} z \xrightarrow{\text{Affine}} s \xrightarrow{\text{Softmax}} y$$

که در آن:

- x : بردار ورودی
- $h = W_1 x + b_1$: خروجی لایه Affine اول
- $\text{ReLU}(h) = \max(0, h)$: به عنوان تابع فعال‌سازی
- $z = W_2 x + b_2$: نگاشت خطی دوم
- s : نمرات کلاس‌ها
- $y = \text{Softmax}(s)$: توزیع احتمالات کلاس‌ها

این معماری پایه، هرچند ساده است، اما بستری مناسب برای تست و رفع خطا در کدنویسی گذر مستقیم (Forward) و گذر معکوس (Backward) فراهم نمود و مرجع مقایسه‌ای برای مدل‌های پیچیده‌تر بعدی محسوب شد.

^۱ Mean Squared Error

۲-۵-۲ مدل تعمیم‌یافته: شبکه عمیق N لایه

به منظور افزایش ظرفیت شبکه و توانایی آن در یادگیری الگوهای غیرخطی پیچیده‌تر، مدل پایه به یک معماری قابل توسعه با تعداد دلخواهی از لایه‌های مخفی تعمیم داده شد. در این طراحی، بلوک محاسباتی استاندارد به صورت تکراری به کار گرفته شد تا امکان تنظیم عمق شبکه متناسب با نیاز مسئله فراهم گردد. ساختار کلی این شبکه به صورت زیر بیان می‌شود:

$$\{\text{Affine} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow [\text{Dropout}]\}^{(L-1)} \rightarrow \text{Affine} \rightarrow \text{Softmax}$$

در این معادله:

بلوک داخل آکولاد به تعداد $(L-1)$ بار تکرار می‌شود (که در آن L تعداد کل لایه‌ها است).

- Batch Normalization (در صورت فعال‌سازی) برای نرمال‌سازی توزیع داده در هر لایه و تسهیل فرآیند همگرایی به کار می‌رود.

- Dropout (اختیاری) به عنوان یکی از روش‌های رگولاریزیشن، بخشی از نورون‌ها را به صورت تصادفی غیرفعال می‌کند تا از بیش‌برازش (Overfitting) جلوگیری شود.

- در نهایت، یک لایه Affine انتهایی، بردار ویژگی لایه آخر را به امتیازات کلاس‌ها نگاشت داده و به Softmax متصل می‌شود.

مزیت اصلی این معماری آن است که به سادگی امکان افزودن یا حذف لایه‌های مختلف (نرمال‌سازی، فعال‌سازی و رگولاریزیشن) وجود دارد. این موضوع نه تنها انعطاف‌پذیری بالایی در طراحی مدل فراهم می‌کند، بلکه امکان تحلیل تجربی تأثیر هر تکنیک بر کیفیت آموزش و عملکرد نهایی شبکه را نیز فراهم ساخت.

۲-۶ فرآیند آموزش و بهینه‌سازی

پس از تعریف معماری مدل‌های شبکه، فرآیند آموزش و بهینه‌سازی پارامترها (وزن‌ها و بایاس‌ها) با هدف کمینه‌سازی تابع زیان آغاز گردید. این فرآیند بر پایه دو جزء کلیدی استوار بود: یک چارچوب مدیریتی برای آموزش و مجموعه‌ای از الگوریتم‌های بهینه‌سازی پیشرفته.

۲-۶-۱ چارچوب آموزش^۱

به منظور جداسازی منطق آموزش از تعریف معماری مدل، یک چارچوب مدیریتی اختصاصی (Solver) پیاده‌سازی شد. این چارچوب، مسئولیت اجرای کامل حلقه آموزش را بر عهده داشت. وظایف اصلی آن شامل موارد زیر بود:

- مدیریت حلقه‌های آموزش^۲: اجرای فرآیند آموزش در تعداد دوره‌های مشخص.
- نمونه‌برداری تصادفی: انتخاب دسته‌های کوچک داده^۳ از مجموعه آموزش در هر تکرار برای اعمال گرادیان کاهشی تصادفی (SGD).
- اجرای گذر مستقیم و معکوس: فراخوانی توابع forward و backward برای محاسبه زیان و گرادیان‌ها.

^۱ Training Solver

^۲ Epochs

^۳ Mini-batches

- به‌روزرسانی پارامترها: استفاده از الگوریتم بهینه‌سازی منتخب برای تغییر وزن‌ها و بایاس‌ها.
- ارزیابی عملکرد: محاسبه و ثبت دقت مدل روی داده‌های آموزش و اعتبارسنجی در بازه‌های زمانی مشخص جهت نظارت بر یادگیری و تشخیص بیش‌برازش.

۲-۶-۲ الگوریتم‌های بهینه‌سازی

برای به‌روزرسانی پارامترهای مدل، علاوه بر روش ساده گرادیان کاهشی، سه الگوریتم پیشرفته مورد استفاده قرار گرفتند:

الف) گرادیان کاهشی با مومنتوم^۱:

این روش با افزودن یک عبارت مومنتوم (اینرسی) به فرآیند به‌روزرسانی، باعث همگرایی سریع‌تر و عبور از کمینه‌های محلی می‌شود. به‌روزرسانی وزن‌ها بر اساس رابطه زیر انجام می‌شود:

$$v_t = \mu v_{t-1} - \alpha \nabla L(W_t) \quad (11-1)$$

$$W_{t+1} = W_t + v_t$$

که در آن v بردار سرعت، μ ضریب مومنتوم و α نرخ یادگیری است.

ب) الگوریتم RMSProp:

RMSProp یک روش تطبیقی است که نرخ یادگیری را برای هر پارامتر به صورت جداگانه تنظیم می‌کند. این الگوریتم با استفاده از میانگین متحرک مربعات گرادیان، نرخ یادگیری را به‌گونه‌ای تغییر می‌دهد که برای گرادیان‌های بزرگ کاهش و برای گرادیان‌های کوچک افزایش یابد:

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1-\gamma)(\nabla L(W_t))^2 \quad (12-1)$$

$$W_{t+1} = W_t - \frac{\alpha}{\sqrt{E[g^2]_t + \epsilon}} \nabla L(W_t)$$

که در آن $E[g^2]_t$ میانگین متحرک مربعات گرادیان و γ ضریب واپاشی است.

ج) الگوریتم Adam^۲:

Adam ترکیبی از دو روش مومنتوم و RMSProp است و از میانگین متحرک گرادیان‌ها (گشتاور اول) و میانگین متحرک مربعات گرادیان‌ها (گشتاور دوم) برای تنظیم نرخ یادگیری استفاده می‌کند. روابط اصلی آن به صورت زیر است:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) \nabla L(W_t) \quad (13-1)$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) (\nabla L(W_t))^2$$

$$W_{t+1} = W_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

^۱ SGD with Momentum

^۲ Adaptive Moment Estimation

که در آن $\hat{m}_t$ و $\hat{v}_t$ نسخه‌های تصحیح شده بایاس گشتاورهای اول و دوم هستند. الگوریتم Adam به دلیل سرعت همگرایی بالا و عملکرد قوی، یکی از محبوب ترین روش‌های بهینه‌سازی در یادگیری عمیق محسوب می‌شود.

۷-۲ تکنیک‌های رگولاریزیشن

برای افزایش قابلیت تعمیم مدل و جلوگیری از پدیده بیش‌برازش، دو تکنیک استاندارد رگولاریزیشن و همچنین نرمال‌سازی بچ در این فریمورک به کار گرفته شد.

۱-۷-۲ رگولاریزیشن L_2

همچنین به عنوان کاهش وزن^۱ شناخته می‌شود، این روش با افزودن یک عبارت جریمه به تابع زیان اصلی از بزرگ شدن بیش از حد وزن‌ها جلوگیری می‌کند:

$$L_{total} = L_{original} + \lambda \sum_i W_i^2 \quad (۱۴-۱)$$

که در آن $L_{original}$ زیان اصلی مدل، W وزن‌های لایه i و λ یک هاپرپارامتر برای کنترل شدت رگولاریزیشن است. کمینه کردن این تابع زیان باعث می‌شود مدل وزن‌های کوچک‌تری انتخاب کند و قابلیت تعمیم بالاتری داشته باشد.

Dropout ۲-۷-۲

Dropout برای جلوگیری از بیش‌برازش و هم‌سازگاری بیش از حد نورون‌ها به کار می‌رود. در طول آموزش، برخی نورون‌ها به صورت تصادفی غیرفعال می‌شوند و نورون‌های باقی‌مانده با ضری $1/p$ مقیاس‌دهی می‌شوند (Inverted Dropout). این کار باعث می‌شود مدل به یادگیری ویژگی‌های مستقل و مقاوم‌تر ترغیب شود و در مرحله تست نیازی به تغییر شبکه نیست.

Dropout ۳-۷-۲

Batch Normalization به پایدارسازی و تسریع فرآیند آموزش کمک می‌کند و به صورت غیرمستقیم اثر رگولاریزیشن جزئی دارد. نوسانات میانگین و واریانس در هر mini-batch باعث می‌شود مدل کمتر به نمونه خاصی وابسته باشد و ویژگی‌های مقاوم‌تری یاد بگیرد.

۳- نتایج

در این بخش، نتایج تجربی به دست آمده از فرآیندهای آموزش و ارزیابی فریمورک پیاده‌سازی شده، ارائه و تحلیل می‌گردد. هدف از این آزمایش‌ها، اعتبارسنجی صحت عملکرد اجزای مختلف فریمورک و همچنین سنجش کارایی مدل نهایی بر روی وظیفه طبقه‌بندی تصاویر دیتاست CIFAR-۱۰ است.

^۱ Weight Decay

ارزیابی‌ها به صورت مرحله‌ای انجام پذیرفت. در ابتدا، صحت پیاده‌سازی گرادین‌ها با استفاده از روش گرادین عددی تأیید شد. سپس، یک مدل پایه دو لایه برای دستیابی به یک خط مبنای عملکردی (Baseline) آموزش داده شد. در ادامه، تأثیر الگوریتم‌های بهینه‌سازی مختلف بر سرعت همگرایی و دقت نهایی مدل مورد مقایسه قرار گرفت. در نهایت، با بهره‌گیری از نتایج این آزمایش‌ها، بهترین هایپرپارامترها برای آموزش یک شبکه عمیق انتخاب و عملکرد نهایی آن بر روی مجموعه داده آزمون گزارش گردید.

۳-۱ اعتبارسنجی صحت پیاده‌سازی

رگونه آموزش، صحت پیاده‌سازی لایه‌های محاسباتی و مدل‌های شبکه از طریق بررسی گرادین عددی^۱ مورد تأیید قرار گرفت. در این روش، گرادین محاسبه‌شده به صورت تحلیلی (از طریق پس‌انتشار) با گرادین تخمین‌زده شده به روش عددی مقایسه می‌شود. نتایج این اعتبارسنجی در جدول ۱ خلاصه شده است.

جدول ۱- نتایج اعتبارسنجی گرادین برای اجزای مختلف فریمورک

خطای نسبی (Relative Error)	لایه / مدل
$9.90e-11$	Affine Layer (Backward Pass)
$3.27e-12$	ReLU Layer (Backward Pass)
$2.53e-07$	Two-Layer Net (reg=۰, ۷)
$1.14e-08$	Fully-Connected Net (reg=۳, ۱۴)

مقادیر بسیار کوچک خطای نسبی (10^{-7}) نشان‌دهنده پیاده‌سازی صحیح و قابل اعتماد محاسبات گذر معکوس در تمام اجزای فریمورک است و اطمینان می‌دهد که فرآیند یادگیری بر پایه محاسبات گرادین درست انجام خواهد شد.

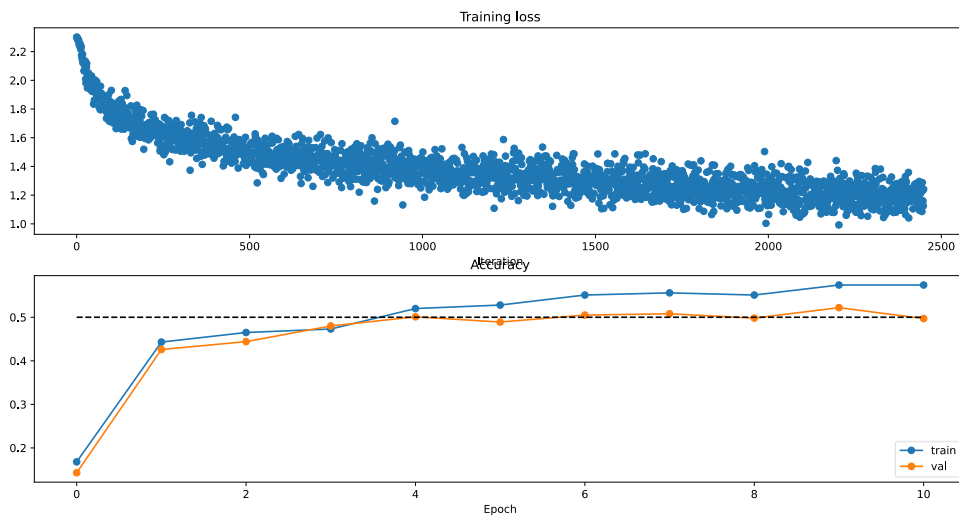
۳-۲ آموزش مدل پایه (شبکه دو لایه)

به عنوان یک خط مبنا (Baseline)، یک شبکه دو لایه با ۱۰۰ نورون در لایه مخفی به مدت ۱۰ تکرار (epoch) بر روی مجموعه داده کامل CIFAR-۱۰ آموزش داده شد. هایپرپارامترهای این آزمایش به شرح زیر بود:

- الگوریتم بهینه‌سازی SGD
- نرخ یادگیری $1e-3$
- اندازه batch: ۲۰۰
- ضریب رگولاریزیشن $L2: 0.001$

که نمودارهای فرآیند آموزش این شبکه در شکل ۲ آورده شده است.

^۱ Numerical Gradient Checking

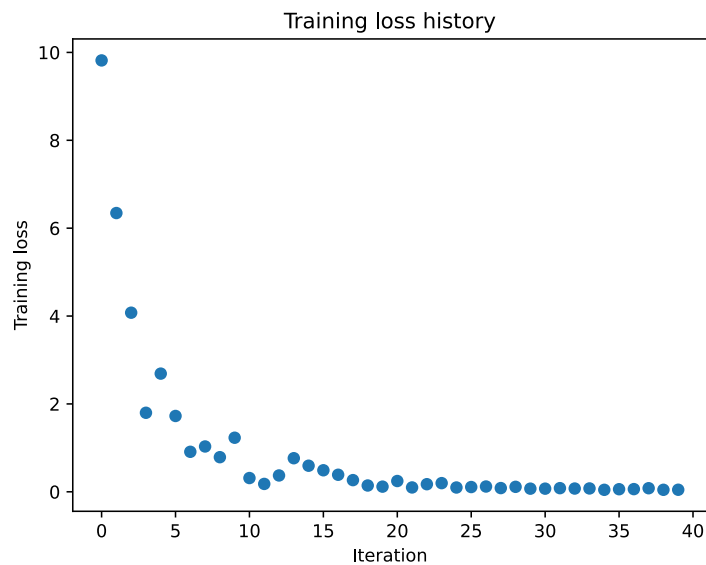


شکل ۲- نمودار تابع زیان (بالا) و دقت آموزش/اعتبارسنجی (پایین) برای شبکه دو لایه.

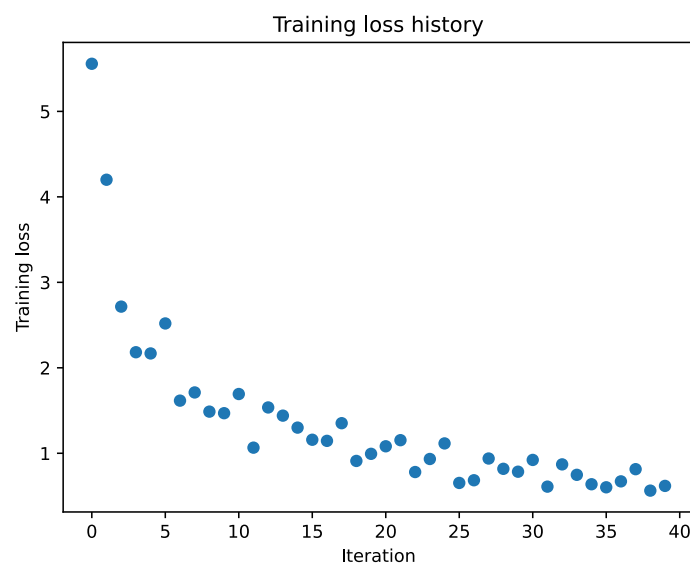
همانطور که مشاهده می‌شود، تابع زیان به طور پیوسته کاهش یافته و دقت مدل بر روی هر دو مجموعه آموزش و اعتبارسنجی به طور پیوسته افزایش می‌یابد. مدل توانست به حداکثر دقت ۵۲.۲٪ بر روی مجموعه داده اعتبارسنجی (در epoch نهم) دست یابد. این نتیجه، هدف اولیه پروژه را برآورده کرده و نشان می‌دهد که فریمورک قادر به یادگیری الگوهای معنادار از داده‌ها می‌باشد.

۳-۳ تحلیل ظرفیت مدل (آزمایش بیش‌برازش)

به منظور اطمینان از اینکه مدل‌های پیاده‌سازی شده دارای ظرفیت کافی برای یادگیری هستند، یک آزمایش بیش‌برازش (Overfitting) بر روی یک زیرمجموعه کوچک از داده‌های آموزشی (۵۰ نمونه) انجام شد. هدف این بود که مدل بتواند به دقت ۱۰۰٪ بر روی این داده‌های محدود دست یابد. این آزمایش برای دو معماری، یک شبکه ۳ لایه و یک شبکه ۵ لایه، انجام شد.



شکل ۳- نمودار تابع زیان برای آزمایش بیش‌برازش یک شبکه ۳ لایه.



شکل ۴- نمودار تابع زیان برای آزمایش بیش‌برازش یک شبکه ۵ لایه.

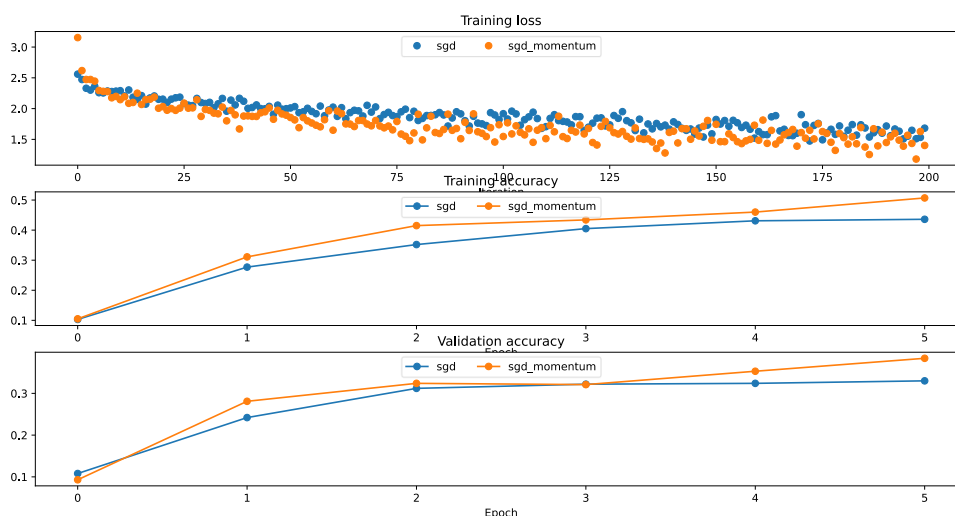
نتایج شبکه ۳ لایه (شکل ۳): با هایپرپارامترهای $\text{learning Rate} = 1e-3$ و $\text{Weight Scale} = 2e-3$ ، تابع زیان به سرعت به سمت صفر میل کرده و دقت آموزش در epoch یازدهم به ۱۰۰٪ رسید. این نتیجه تأیید می‌کند که مدل دارای ظرفیت کافی برای یادگیری کامل داده‌های آموزشی است.

نتایج شبکه ۵ لایه (شکل ۴): با هایپرپارامترهای $\text{learning Rate} = 1e-3$ و $\text{Weight Scale} = 2e-5$ مدل توانست به دقت نهایی ۹۲٪ در آموزش دست یابد. اگرچه به دقت کامل نرسید، اما فرآیند یادگیری

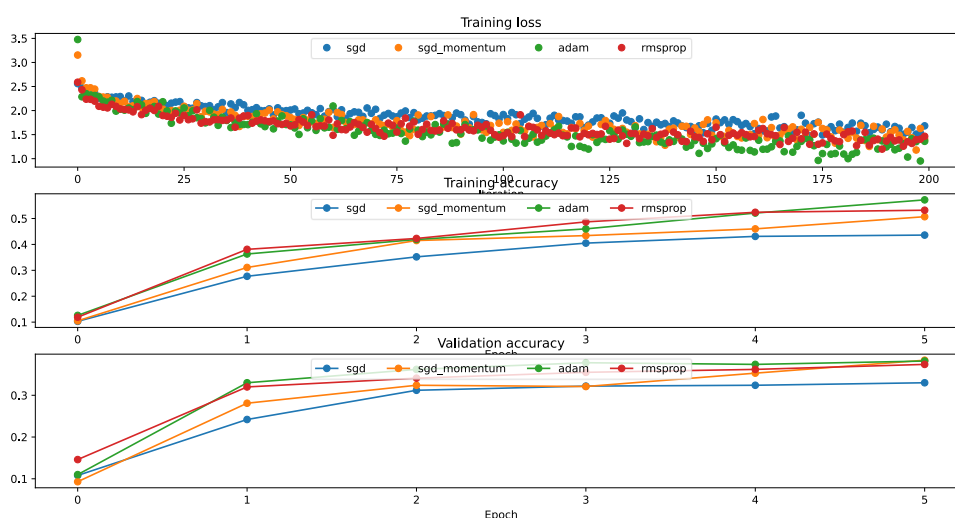
موفقیت‌آمیز بود و نشان داد که شبکه‌های عمیق‌تر برای رسیدن به همگرایی کامل به تنظیم دقیق‌تری نیاز دارند.

۳-۴ مقایسه عملکرد الگوریتم‌های بهینه‌سازی

برای ارزیابی تأثیر الگوریتم‌های بهینه‌سازی مختلف، یک شبکه عمیق ۶ لایه (۵ لایه مخفی با ۱۰۰ نورون) به مدت ۵ دوره با چهار الگوریتم SGD, SGD+Momentum, RMSProp و Adam آموزش داده شد. هاپرپارامترهای مشترک شامل $\text{Batch Size} = 100$ و $\text{Weight Scale} = 5e - 2$ بودند. نرخ یادگیری برای هر الگوریتم به صورت جداگانه تنظیم شد. نتایج در شکل‌های ۵ و ۶ مقایسه شده‌اند.



شکل ۵- مقایسه عملکرد SGD و SGD+Momentum



شکل ۶- مقایسه عملکرد چهار الگوریتم بهینه‌سازی

نتایج عددی نهایی برای هر الگوریتم در جدول ۲ خلاصه شده است:

جدول ۲: مقایسه دقت نهایی الگوریتم‌های بهینه‌سازی پس از ۵ دوره.

الگوریتم بهینه‌سازی	نرخ یادگیری	دقت نهایی اعتبار سنجی
SGD	$1e-2$	۳۳٪
SGD+Momentum	$1e-2$	۳۸.۴٪
RMSProp	$1e-4$	۳۷.۴٪
Adam	$1e-3$	۳۸.۲٪

نتایج به وضوح نشان می‌دهد که SGD+Momentum عملکرد SGD ساده را به طور قابل توجهی بهبود می‌بخشد. همچنین، الگوریتم‌های تطبیقی (Adam و RMSProp) همگرایی بسیار سریع‌تری دارند و در همان دور اول به دقت بالایی می‌رسند. در این آزمایش، SGD+Momentum با اختلاف جزئی، بهترین عملکرد را در پایان دوره پنجم از خود نشان داد.

۳-۵ نتایج مدل نهایی

با بهره‌گیری از نتایج آزمایش‌های پیشین، یک مدل نهایی با هدف دستیابی به بالاترین دقت ممکن آموزش داده شد. هایپرپارامترهای این مدل بر اساس مشاهدات قبلی (مانند برتری الگوریتم Adam در همگرایی سریع) بهینه شدند.

- معماری: شبکه ۳ لایه (با دو لایه مخفی با ۲۰۰ نورون در هر کدام)
- الگوریتم بهینه‌سازی Adam
- نرخ یادگیری $1e-3$
- اندازه batch: ۲۵۶
- ضریب رگولاریزیشن L2 : ۰.۰۱
- تعداد دوره‌های آموزش: ۲۰۰

پس از اتمام فرآیند آموزش، عملکرد نهایی مدل بر روی مجموعه داده اعتبارسنجی و آزمون ارزیابی گردید.

جدول ۳- دقت نهایی بهترین مدل بر روی مجموعه داده اعتبارسنجی و آزمون

مجموعه داده	دقت نهایی
اعتبارسنجی	۵۲.۷٪
آزمون	۵۳.۰٪

مدل نهایی توانست به دقت قابل قبولی دست یابد که نشان‌دهنده کارایی فریمورک پیاده‌سازی شده در حل یک مسئله طبقه‌بندی تصویر واقعی است. اگرچه در آزمایش مقایسه‌ای کوتاه-مدت، الگوریتم

SGD+Momentum به دقت نهایی اندکی بالاتر دست یافت، اما الگوریتم Adam به دلیل همگرایی بسیار سریع تر در دوره‌های اولیه و پایداری بیشتر در تنظیم هایپر پارامترها به عنوان بهینه‌ساز نهایی انتخاب گردید. نتایج نزدیک به هم در دو مجموعه اعتبارسنجی و آزمون، نشان‌دهنده قابلیت تعمیم خوب مدل و عدم وقوع بیش‌برازش قابل توجه است.

نتیجه‌گیری

در این پروژه، یک فریمورک نرم‌افزاری برای ساخت و آموزش شبکه‌های عصبی تمام‌متصل با معماری ماژولار با موفقیت از پایه پیاده‌سازی شد. هدف اصلی این پروژه، که همانا درک عمیق اجزای داخلی یک شبکه عصبی بود، از طریق پیاده‌سازی دستی لایه‌های محاسباتی، الگوریتم‌های بهینه‌سازی و یک چارچوب آموزشی جامع، محقق گردید. فرآیند اعتبارسنجی گرادینان‌ها صحت محاسبات پس‌انتشار را تأیید کرد و آزمایش‌های تجربی بر روی دیتاست CIFAR-10، کارایی عملی فریمورک را به اثبات رساند.

یافته‌های کلیدی این پروژه را می‌توان به شرح زیر خلاصه کرد:

- ❖ اثربخشی معماری ماژولار: طراحی ماژولار به ما اجازه داد تا به سادگی معماری‌های مختلف را با ترکیب لایه‌های پایه ایجاد و آزمایش کنیم، که این امر اهمیت تفکیک مسئولیت‌ها در طراحی نرم‌افزارهای یادگیری عمیق را برجسته می‌سازد.
- ❖ برتری بهینه‌سازهای تطبیقی: نتایج مقایسه الگوریتم‌های بهینه‌سازی به وضوح نشان داد که روش‌های تطبیقی مانند Adam، به دلیل همگرایی سریع‌تر و پایداری بیشتر، گزینه‌های کارآمدتری نسبت به SGD ساده برای آموزش شبکه‌های عمیق هستند.
- ❖ دستیابی به عملکرد قابل قبول: مدل نهایی توانست به دقت ۵۳.۰٪ بر روی مجموعه آزمون CIFAR-10 دست یابد. اگرچه این نتیجه با مدل‌های پیشرفته امروزی فاصله دارد، اما برای یک شبکه تمام‌متصل که از پایه پیاده‌سازی شده، یک دستاورد قابل توجه محسوب می‌شود و نشان‌دهنده قابلیت فریمورک در حل مسائل واقعی است.
- با وجود موفقیت‌های کسب شده، این پروژه زمینه‌هایی برای بهبود و گسترش در آینده دارد. به عنوان کارهای آینده، می‌توان موارد زیر را پیشنهاد داد:
- ❖ گسترش به شبکه‌های کانولوشنی^۱: افزودن لایه‌های کانولوشنی و پولینگ به فریمورک می‌تواند قابلیت آن را برای حل مسائل بینایی کامپیوتر به طور چشمگیری افزایش دهد.
- ❖ پیاده‌سازی تکنیک‌های رگولاریزیشن پیشرفته‌تر: افزودن روش‌هایی مانند Batch و Dropout Normalization (که در این پروژه نیز پیاده‌سازی شدند) می‌تواند به بهبود بیشتر قابلیت تعمیم مدل کمک کند.
- ❖ بهینه‌سازی عملکرد محاسباتی: جایگزینی پیاده‌سازی‌های ساده و مبتنی بر حلقه (Naive) با نسخه‌های برداری شده و کارآمدتر می‌تواند سرعت آموزش را به میزان قابل توجهی افزایش دهد.

^۱ CNNs

در مجموع، این پروژه یک تجربه آموزشی ارزشمند بود که درک عمیقی از مبانی نظری و چالش‌های عملی در پیاده‌سازی سیستم‌های یادگیری عمیق فراهم آورد.

Surname: Mirzajani	Name: Amirreza
Title: Implementation of a Modular Framework for Fully-Connected Neural Networks and CIFAR-۱۰ Image Classification	
Supervisor: Dr. M. Shoaran	
Degree: Master of Science	Major: Mechanical Engineering Field: Applied Design
University: University of Tabriz	Faculty: Mechanical Engineering
Pages: ۱۸	
Keywords: Fully-Connected Neural Network, Deep Learning, Modular Architecture, Backpropagation, Adam Optimizer	
Abstract: In this project, a software framework for building and training fully-connected neural networks with a modular architecture was implemented from scratch. The primary objective was to gain a deep understanding of the internal components of a neural network, from fundamental computational layers to advanced optimization algorithms. This framework includes the implementation of linear (affine) layers, ReLU activation functions, a comprehensive training framework (Solver), and modern optimization algorithms such as SGD+Momentum and Adam. The correctness of the framework's functionality was verified through numerical gradient checking. Subsequently, the framework was employed to train and evaluate models on the CIFAR-۱۰ image classification task. Experimental results demonstrated that adaptive optimization algorithms like Adam achieve faster convergence compared to vanilla SGD. The final model, trained with optimized hyperparameters, achieved a final accuracy of ۵۳,۰٪ on the test set, proving the framework's practical effectiveness in solving a real-world problem.	



University of Tabriz

**Faculty of Electrical and Computer Engineering
Department of Computer Engineering**

Title:

Implementation of a Modular Framework for Fully-Connected Neural Networks and
CIFAR-۱۰ Image Classification

Professor:

Dr. M. Shoaran

Researcher:

Amirreza Mirzajani

August ۲۰۲۰